XRPL Validator — Daily Engineering Report · 2026-10-28

Period: 2026-10-28 00:00 to 24:00 US Eastern (2026-10-28 04:00 to 2026-10-29 04:00 UTC); ledgers #107,950,580 to #107,972,241. Every count from the ledger and the validator's logs stops at #107,972,241, closed 03:59:51 UTC. Generated 2026-10-29 00:20 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Status

Normal — coverage outside planned maintenance 99.968% (objective 99.9%), every validation agreed with the network, no open engine receipt.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** signed 21,613 of the 21,662 ledgers the network closed (99.774%); every validation matched the network's validated ledger. Unsigned: 42 during planned restarts, 7 outside maintenance (1 incident).
- **Native engine:** 0 engine receipts.
- **Changes:** 1 deployment (cycle 190), 22 commits merged into main.
- **Network:** 1 slow-close period; the longest, 04:05 UTC–04:05 UTC, closed ledgers up to 18 s apart.
- **Independence program:** 0 of 30 qualifying days (snapshot at 04:20:42 UTC).
- **Amendment fixCleanup3_4_0** holds majority; activation due ~2026-11-04 14:46 UTC.

Service levels

Measure	Value
Ledgers closed by the network	21,662
Validations issued	21,613
Coverage (validations / ledgers closed)	99.774%
Validations matching the network's validated ledger hash	21,613 of 21,613
Unsigned during planned restarts	42
Unsigned outside maintenance	7
Coverage outside planned maintenance (objective 99.9%)	99.968%
Restarts	1
Basis: our own ledger hash	21,606
Basis: our own hash, witnessed by the reference node (quorum short)	7

Measure	Value
External agreement, data.xrpl.org (UTC day 2026-10-28)	0.99723 (55 missed of 19,825)

Restarts:

- 22:52:28 UTC: deployment of cycle 190; validations resumed at #107,967,667 (42 ledgers unsigned between the last validation before the restart and the first after it)

The basis rows sum to validations issued. Coverage counts the validations the validator issued for ledgers the network closed in the period. Agreement compares each validation's ledger hash with the network's validated hash for that ledger. data.xrpl.org scores UTC days from the validations its crawlers received, so it can differ.

Native engine conformance

Ledgers with a receipt in the period (#107,950,580–#107,972,241), by step:

Step	Ledgers with a receipt
Step 1 · native engine	0
Step 2 · records	0
Step 3 · order	0
Step 4 · peer data	0
Step 4 · validations	0
Step 5 · own hash	0
Writer	0

Live counters, snapshot at 04:20:42 UTC (since the last restart, 2026-10-28 22:52 UTC; not cut off at the period's last ledger):

Step	Checked	Result
1 · native engine vs libxrpl	4,852 ledgers, 894,113 transactions	4,852 fully matched; results 0, bytes 0, keys 0/0
2 · transaction records	4,880 ledgers, 902,204 records	902,204 exact; differ 0, tree 0
3 · transaction order	4,914 ledgers (mode: on)	4,914 exact; order 0, results 0, bytes 0, refused 0
4 · ledger data from peers	4,928 ledgers fetched	0 matched; differ 0, failed 0
4 · trusted validations	4,926 ledgers judged	4,926 agreed; differ 0, short of quorum 0
5 · our own ledger hash	4,928 ledgers	4,880 own; differ 0, partial 48
Writer	every ledger written	mismatch 0, distrusted 0

"Partial" ledgers are those the engine could not yet build alone while warming up after a restart.

Receipt register

No receipts this day.

Incidents

INC-01 · 7 ledgers unsigned

Window	22:57:00 UTC to 22:57:21 UTC
Ledgers	#107,967,693–#107,967,699 (7)
Impact	the validator issued no validation for these ledgers; no incorrect validation was issued
Cause	Cause not recorded.

Change log

Deployments

Cycle	Main	Deployed	Test suites	Replay windows	Post-deploy health	Soak
190	`ac262cd`	22:53:22 UTC	2,212 passed / 0 failed; port 1,006 / 0	HIST60 60 ledgers, 0 mismatches, WIN98 98 ledgers, 0 mismatches	389 matched, 0 mismatched	#65 from 23:19 UTC: in progress

Cycle 190 delivered: `ddb157a` fix(meta): a Batch inner that writes an object back unchanged marks it written, so a line deleted before the issuer's root gives it FinalFields; `54c8c94` docs(step4): the peer switch plan — ledgers from peers, a non-signing twin to prove it; `50c6396` feat(step4): XRPL_LEDGER_SOURCE switch; peer mode refuses and counts every RPC call; `8b1e893` feat(step4): own_validated says when a newer ledger reaches quorum; `6a0cfe4` feat(step4): peer_fetch serves awaited ledger, header and state jobs; keyed state fetch; `0d9d2c8` feat(step4): peer_ledgers — the validated hash for any seq and its inputs from peers; `7815dcc` feat(step4): the native writer hands over its own system-object writes; `6dfc72e` feat(step4): the validator's ledger feed takes each ledger from peers when XRPL_LEDGER_SOURCE=peers; `cd02a54` feat(step4): warm restart from peers; the last the reference node reads go; `3d260b5` test(step4): the quorum wait tests run on real time (no tokio test-util); `1ca6783` feat(step4): XRPL_WITNESS_RPC and XRPL_OBSERVE; /api/engine shows the source; `c99ecad` test(step4): the observe test installs rustls's provider as live_viewer does; `16fb48b` feat(step8): cold start from peers — proven leaves become a store live_viewer resumes; `0df600c` fix(paths): every receipt and state file the validator writes follows XRPL_DATA_DIR; `1c079ba` feat(step4): the twin — a peer-fed, non-signing copy and its check; `08d813e` fix(step4): the build without ffi compiles its tests again; one stream closure becomes a fn; `74abea4` fix(step4): review findings — no FFI repair RPC from peers, a warm stop never waits on a peer fetch, peers need the writer; `cc1d4bc` docs(step4): the final review's findings and fixes; `f2083dd` fix(step8): the peer cold start takes the live launcher's list pin and runs at the lowest priority; `4fcdd98` fix(step4): from peers the mirror loads at once, however far behind; `c63bd2e` feat(5a): XRPL_OWN_BREAK — the breaker's wait, or off for the 30-day count; `ac262cd` feat(step4): twin-watch.sh — the twin check once an hour, to the twin's data dir

Each deployment passes the unit and port test suites and replays mainnet history windows with zero mismatches before it ships, is health-checked after it starts, and then soaks for 24 hours.

Merged into main

Merged	Repo	Commit	Subject
17:09 UTC	xrpl-validator	`ac262cd`	feat(step4): twin-watch.sh — the twin check once an hour, to the twin's data dir
17:09 UTC	xrpl-validator	`c63bd2e`	feat(5a): XRPL_OWN_BREAK — the breaker's wait, or off for the 30-day count

Merged	Repo	Commit	Subject
17:09 UTC	xrpl-validator	`4fcdd98`	fix(step4): from peers the mirror loads at once, however far behind
17:09 UTC	xrpl-validator	`f2083dd`	fix(step8): the peer cold start takes the live launcher's list pin and runs at the lowest priority
17:09 UTC	xrpl-validator	`cc1d4bc`	docs(step4): the final review's findings and fixes
17:09 UTC	xrpl-validator	`74abea4`	fix(step4): review findings — no FFI repair RPC from peers, a warm stop never waits on a peer fetch, peers need the writer
17:09 UTC	xrpl-validator	`08d813e`	fix(step4): the build without ffi compiles its tests again; one stream closure becomes a fn
17:09 UTC	xrpl-validator	`1c079ba`	feat(step4): the twin — a peer-fed, non-signing copy and its check
17:09 UTC	xrpl-validator	`0df600c`	fix(paths): every receipt and state file the validator writes follows XRPL_DATA_DIR
17:09 UTC	xrpl-validator	`16fb48b`	feat(step8): cold start from peers — proven leaves become a store live_viewer resumes
17:09 UTC	xrpl-validator	`c99ecad`	test(step4): the observe test installs rustls's provider as live_viewer does
17:09 UTC	xrpl-validator	`1ca6783`	feat(step4): XRPL_WITNESS_RPC and XRPL_OBSERVE; /api/engine shows the source
17:09 UTC	xrpl-validator	`3d260b5`	test(step4): the quorum wait tests run on real time (no tokio test-util)
17:09 UTC	xrpl-validator	`cd02a54`	feat(step4): warm restart from peers; the last the reference node reads go
17:09 UTC	xrpl-validator	`6dfc72e`	feat(step4): the validator's ledger feed takes each ledger from peers when XRPL_LEDGER_SOURCE=peers
17:09 UTC	xrpl-validator	`7815dcc`	feat(step4): the native writer hands over its own system-object writes
17:09 UTC	xrpl-validator	`0d9d2c8`	feat(step4): peer_ledgers — the validated hash for any seq and its inputs from peers
17:09 UTC	xrpl-validator	`6a0cfe4`	feat(step4): peer_fetch serves awaited ledger, header and state jobs; keyed state fetch
17:09 UTC	xrpl-validator	`8b1e893`	feat(step4): own_validated says when a newer ledger reaches quorum
17:09 UTC	xrpl-validator	`50c6396`	feat(step4): XRPL_LEDGER_SOURCE switch; peer mode refuses and counts every RPC call
17:09 UTC	xrpl-validator	`54c8c94`	docs(step4): the peer switch plan — ledgers from peers, a non-signing twin to prove it
17:09 UTC	xrpl-validator	`ddb157a`	fix(meta): a Batch inner that writes an object back unchanged marks it written, so a line deleted before the issuer's root gives it FinalFields

Findings fixed

No engine findings merged into main this day.

Other engineering changes

- **feat(step4): twin-watch.sh — the twin check once an hour, to the twin's data dir** (`ac262cd`, xrpl-validator)

- **feat(5a): XRPL_OWN_BREAK — the breaker's wait, or off for the 30-day count** (`c63bd2e` , xrpl-validator) The breaker signed the quorum's hash after 15 min with nothing of our own signed. For the count a borrowed signature costs the whole day while an unsigned ledger costs only coverage (2% a day), so XRPL_OWN_BREAK=off leaves what we cannot make unsigned. Default unchanged (900 s). The twin runs it off; its first run showed why (a 16-minute restart and catch-up tripped the breaker).
- **fix(step4): from peers the mirror loads at once, however far behind** (`4fcdd98` , xrpl-validator) Until the native mirror is in, each ledger lands by the keyed state fetch (about ten peer requests), and the twin's first run measured 3 ledgers a minute against the network's 15: it could never get within the old two-ledger gate to start loading. In peer mode the load starts at once; ledgers landed meanwhile queue and replay as before, then the catch-up is native (about three requests a ledger).
- **fix(step8): the peer cold start takes the live launcher's list pin and runs at the lowest priority** (`f2083dd` , xrpl-validator)
- **fix(step4): review findings — no FFI repair RPC from peers, a warm stop never waits on a peer fetch, peers need the writer** (`74abea4` , xrpl-validator)
- ffi_engine: the tefBAD_LEDGER repair loop and the terPRE_SEQ diagnostic/repair built their own RpcProvider and read the reference node past the consult budget and the refusal count. Off in peer mode (the FFI leg is a comparison only). - ws_sync: the ledger's fetch from peers now runs before the warm-stop gate (nothing is written while it comes), and the keyed state fetch inside the gate gives up when a stop is requested …
- **fix(step4): the build without ffi compiles its tests again; one stream closure becomes a fn** (`08d813e` , xrpl-validator) The two system-object tests use stage3 (ffi only) and sat in the validator's ledger feed's ungated undo_tests module: gated now. fallback_keys is ffi-only in use. The peer trigger's message closure is ledger_closed(), allowing clippy's result_large_err there (the Err type is the WebSocket's, shared by both sources).
- **feat(step4): the twin — a peer-fed, non-signing copy and its check** (`1c079ba` , xrpl-validator) scripts/twin/run-twin.sh DATA_DIR BINARY: the live engine switches plus XRPL_LEDGER_SOURCE=peers, XRPL_OBSERVE=1, its own XRPL_DATA_DIR and API port 3787, RPC/WS pointed at a closed local port, 40 peers, memory-capped (systemd-run scope, TWIN_MEMORY_MAX default 52G), nice 19 / ionice idle; refuses the live data dir or a dir with a seed. scripts/twin/twin_check.py: twin vs live /api/engine in one line (rpc_refused, …
- **fix(paths): every receipt and state file the validator writes follows XRPL_DATA_DIR** (`0df600c` , xrpl-validator) consensus_rounds, native_meta, independence_days, native_order, unl-last-good, native_shadow (and the writer log and canary beside it), own_hash, own_validated and peer_fetch defaulted to fixed [local path] paths. They now default under paths::data_dir(), which is [local path] unless XRPL_DATA_DIR is set: the live validator (no XRPL_DATA_DIR) is unchanged, and a twin with its own XRPL_DATA_DIR can never write …
- **feat(step8): cold start from peers — proven leaves become a store live_viewer resumes** (`16fb48b` , xrpl-validator) cold_start::convert: a peer_state_sync run (proven.json, header.bin, leaves.bin) is checked again (the header hashes to the ledger hash and names the account hash; the leaves recompute to it, none repeated), then written as state.rocks with the meta:last_seq bookmark, the resume ticket, clean_shutdown.marker and sync_complete.marker; never over an existing store. Bin leaves_to_rocks; peer_state_sync now also writes …
- **feat(step4): XRPL_WITNESS_RPC and XRPL_OBSERVE; /api/engine shows the source** (`1ca6783` , xrpl-validator) witness: with peers as the source, an optional server (XRPL_WITNESS_RPC) checks each landed ledger's hash on its own task: agree/differ/unreachable, and its word goes to own_validated's judging. It never

feeds the data path, so switching it off costs nothing. observe (XRPL_OBSERVE=1, for the twin): a fresh validator identity, the seed file never opened; no relay hub sessions, no manifest broadcast, and at the …

- **feat(step4): warm restart from peers; the last the reference node reads go** (`cd02a54` , xrpl-validator) resume::PeerNetwork: the validated ledger is our own newest quorum ledger (waiting up to 120 s after a start), a ledger's account hash comes from its header fetched from peers by the quorum's (or the parent chain's) hash. live_viewer uses it in peer mode; a start that is not a warm resume exits 75 with a pointer to the peer cold start, never the the reference node bulk sync. FFI leg in peer mode: amendments from our own …
- **feat(step4): the validator's ledger feed takes each ledger from peers when XRPL_LEDGER_SOURCE=peers** (`6dfc72e` , xrpl-validator) The trigger is our own newest quorum ledger (peer_ledgers::trigger, said as ledgerClosed so the loop body is unchanged); no WebSocket, no RPC watchdog poller. Per ledger: header, state root, transactions and records, key sets and wallet delta from peer_ledgers::ledger (an Err holds position, like a failed metadata fetch). The FFI leg takes its blobs from the pairs and skips only the extra root check that read the reference node's …
- **feat(step4): the native writer hands over its own system-object writes** (`7815dcc` , xrpl-validator) NativeShadow::singleton_writes(seq): after propose, the system objects (skip list, the 256-ledger hash list, negative UNL, amendments, fees) our apply changed, as canonical bytes. In peer mode the validator's ledger feed writes these instead of fetching the network's from the reference node; the state hash still checks them.
- **feat(step4): peer_ledgers — the validated hash for any seq and its inputs from peers** (`0d9d2c8` , xrpl-validator) Chain::hash_for: the quorum's hash when the ledger reached one here, else walked back by parent hash from the nearest newer known ledger, each header proven (sequence and hash). ledger(seq): fetch_ledger, assemble, plus the validator's ledger feed's object key sets and wallet delta (extract_affected_nodes / count_wallet_delta, now pub(crate)). trigger(): the newest quorum ledger as it rises (own_validated::quorum_moved, a 1 s look as a …
- **feat(step4): peer_fetch serves awaited ledger, header and state jobs; keyed state fetch** (`6a0cfe4` , xrpl-validator) peer_fetch's driver now runs jobs: Stage A's compare (unchanged: one at a time, the queue capped at 4, oldest dropped) and awaited jobs for Step 4's source, never dropped, up to MAX_ACTIVE (4) at once, each answered on a oneshot: fetch_ledger (header proven to the hash and of that sequence, plus the transaction tree), fetch_header (liBASE alone, 15 s) and fetch_state_keys. Replies route by ledger hash and kind. The …
- **feat(step4): own_validated says when a newer ledger reaches quorum** (`8b1e893` , xrpl-validator) Tally::newest_quorum / newest_quorum(): the highest seq at quorum on one hash. quorum_moved() (tokio Notify, one permit) wakes Step 4's peer source when it rises.
- **feat(step4): XRPL_LEDGER_SOURCE switch; peer mode refuses and counts every RPC call** (`50c6396` , xrpl-validator) ledger_source: rpc (default, unchanged) or peers. In peer mode RippledClient refuses every request and primary_server_info, counts each in ledger_source::stats().rpc_refused, and never moves the WebSocket to a public fallback. peer_fetch::is_source() now follows the switch, so the independence tally sees it. Clippy baseline (ffi, xrpl-node, e595edc): 376 warnings.
- **fix(meta): a Batch inner that writes an object back unchanged marks it written, so a line deleted before the issuer's root gives it FinalFields** (`ddb157a` , xrpl-validator) Step 2 receipts #107920533 67F550DD42ED and #107920565 60700BDAEF38 (each with its tree line and a Step 5 own-hash miss): Batch inners from rU21i7Fh redeem an IOU to its issuer and delete the line. Payment writes the destination back unchanged (rippled's view().update(sleDst), de5fd94), so rippled's inner table holds the issuer's root as modified; the deleted line sorts before it and threadOwners stamps it first, so …

Tests and campaigns

- **test(step4): the observe test installs rustls's provider as live_viewer does** (`c99ecad`, xrpl-validator)
- **test(step4): the quorum wait tests run on real time (no tokio test-util)** (`3d260b5`, xrpl-validator)

Docs and operations

- **docs(step4): the final review's findings and fixes** (`cc1d4bc`, xrpl-validator)
- **docs(step4): the peer switch plan — ledgers from peers, a non-signing twin to prove it** (`54c8c94`, xrpl-validator)

Independence program

Qualifying days: **0 of 30** (snapshot at 04:20:42 UTC). A UTC day qualifies when every validation is of a ledger our own engine built, with our own transaction order and ledger data taken from peers, and coverage is at least 98%.

UTC day 2026-10-28 — tracked #107,947,016–#107,968,633	Value
Validations	21,569 (21,569 own, 0 borrowed)
Coverage of the tracked ledgers	99.77%
Our own transaction order	on
Ledger data from peers	no

Not qualifying: the ledger data came from the reference node, not peers (Step 4).

The validator saves this record every 64 validations, so a day in progress lags the signing log slightly; it is not cut off at the period's last ledger.

Network

Measure	Value
Ledgers closed	21,662 (#107,950,580–#107,972,241)
Mean close interval	3.99 s
Longest close interval	18 s (before #107,950,657)
Closes at least 15 s apart	1
Empty ledgers	0

- Slow closes 04:05:02 UTC–04:05:20 UTC: ledgers #107,950,657, 1 close at least 15 s apart, up to 18 s.

From ledger close times, which the network records at a 10-second resolution or coarser, so single intervals are approximate.

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Amendment	Majority since	Activation due
fixCleanup3_4_0	2026-10-21 14:46 UTC	2026-11-04 14:46 UTC

Amendments enabled on mainnet: 97.

Open items

- INC-01 (#107,967,693–#107,967,699 (7)): cause not recorded.
- Soak #65 (cycle 190) in progress since 2026-10-28 23:19 UTC; due to complete 2026-10-29 23:19 UTC.
- Amendment fixCleanup3_4_0: activation due ~2026-11-04 14:46 UTC.

Corrections to earlier reports

None.

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
10-27 22:48	10-28 17:09	xrpl-validator	`ac262cd`	feat(step4): twin-watch.sh — the twin check once an hour, to the twin's data dir
10-27 22:46	10-28 17:09	xrpl-validator	`c63bd2e`	feat(5a): XRPL_OWN_BREAK — the breaker's wait, or off for the 30-day count
10-27 22:21	10-28 17:09	xrpl-validator	`4fcdd98`	fix(step4): from peers the mirror loads at once, however far behind
10-27 21:48	10-28 17:09	xrpl-validator	`f2083dd`	fix(step8): the peer cold start takes the live launcher's list pin and runs at the lowest priority
10-27 21:42	10-28 17:09	xrpl-validator	`cc1d4bc`	docs(step4): the final review's findings and fixes
10-27 21:41	10-28 17:09	xrpl-validator	`74abea4`	fix(step4): review findings — no FFI repair RPC from peers, a warm stop never waits on a peer fetch, peers need the writer
10-27 21:26	10-28 17:09	xrpl-validator	`08d813e`	fix(step4): the build without ffi compiles its tests again; one stream closure becomes a fn
10-27 21:24	10-28 17:09	xrpl-validator	`1c079ba`	feat(step4): the twin — a peer-fed, non-signing copy and its check
10-27 21:22	10-28 17:09	xrpl-validator	`0df600c`	fix(paths): every receipt and state file the validator writes follows XRPL_DATA_DIR
10-27 21:21	10-28 17:09	xrpl-validator	`16fb48b`	feat(step8): cold start from peers — proven leaves become a store live_viewer resumes
10-27 21:19	10-28 17:09	xrpl-validator	`c99ecad`	test(step4): the observe test installs rustls's provider as live_viewer does
10-27 21:19	10-28 17:09	xrpl-validator	`1ca6783`	feat(step4): XRPL_WITNESS_RPC and XRPL_OBSERVE; /api/engine shows the source
10-27 21:16	10-28 17:09	xrpl-validator	`3d260b5`	test(step4): the quorum wait tests run on real time (no tokio test-util)

Committed	Merged into main	Repo	Commit	Subject
10-27 21:16	10-28 17:09	xrpl-validator	`cd02a54`	feat(step4): warm restart from peers; the last the reference node reads go
10-27 21:13	10-28 17:09	xrpl-validator	`6dfc72e`	feat(step4): the validator's ledger feed takes each ledger from peers when XRPL_LEDGER_SOURCE=peers
10-27 21:09	10-28 17:09	xrpl-validator	`7815dcc`	feat(step4): the native writer hands over its own system-object writes
10-27 21:08	10-28 17:09	xrpl-validator	`0d9d2c8`	feat(step4): peer_ledgers — the validated hash for any seq and its inputs from peers
10-27 21:05	10-28 17:09	xrpl-validator	`6a0cfe4`	feat(step4): peer_fetch serves awaited ledger, header and state jobs; keyed state fetch
10-27 21:01	10-28 17:09	xrpl-validator	`8b1e893`	feat(step4): own_validated says when a newer ledger reaches quorum
10-27 21:00	10-28 17:09	xrpl-validator	`50c6396`	feat(step4): XRPL_LEDGER_SOURCE switch; peer mode refuses and counts every RPC call
10-27 20:54	10-28 17:09	xrpl-validator	`54c8c94`	docs(step4): the peer switch plan — ledgers from peers, a non-signing twin to prove it
10-26 21:37	10-28 17:09	xrpl-validator	`ddb157a`	fix(meta): a Batch inner that writes an object back unchanged marks it written, so a line deleted before the issuer's root gives it FinalFields

Times in UTC. A commit appears on the day it was committed to main or merged into it.